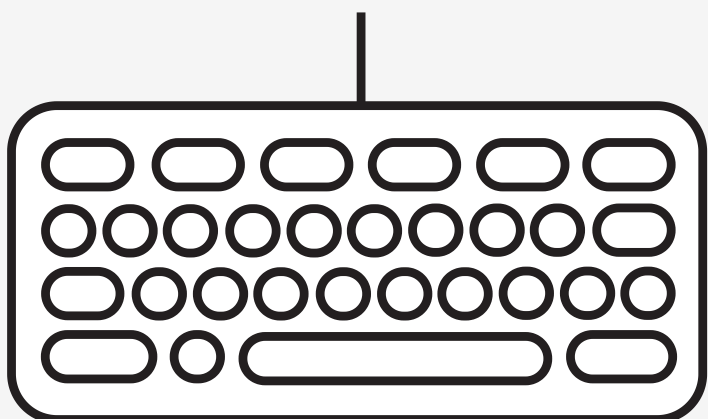




10 Advanced Python Tuple() Interview Questions

THAT TRICK EVEN
EXPERIENCED
DEVELOPERS



Q1. Why can this silently hide bugs in production?

```
data = (1, 2, 3)
a, b, *_ = data
print(a, b)
```

Output

1 2

Why?

- Star-unpacking can silently discard data.
- If an API schema changes:
 - Code still runs
 - Bugs go unnoticed
- **Senior engineers recognize this as a design risk, not a feature.**

Q2. Why does this tuple appear to be mutable?

```
t = ([],)  
t[0].append(1)  
print(t)
```

Output
([1],)

Why?

- Tuples are immutable only at the structure level.
- They do not protect the mutability of the objects they reference.
- This immediately breaks the common myth:
 - “Tuples are immutable.”
 - They are only shallowly immutable.

Q3. If the above Q2 worked, why does this fail?

```
t = ([],)  
t[0] += [1]
```

Output

TypeError: 'tuple' object does not support item assignment

Why?

- Execution order matters:
 - a. += mutates the list
 - b. Python then tries to assign the result back to t[0]
 - c. Tuple item assignment is illegal → error
- Important insight:
 - The mutation already happened, but rebinding fails.

Q4. Then why does this NOT raise any error?

```
t = ([],)  
t += ([1],)  
print(t)
```

Output
([], [1])

Why?

- += on a tuple does not mutate it.
- Instead:
 - A new tuple is created
 - t is rebound to the new object
- No item assignment → no error.
- This distinction separates rebinding vs mutation.

Q5. Can a tuple silently share state across function calls?

```
def f(t=([],)):
    t[0].append(1)
    return t
```

```
print(f())
print(f())
```

Output

```
([1],)
([1, 1],)
```

Why?

- Default arguments are evaluated once at function definition time.
- The tuple is immutable, but it contains a shared mutable list.
- This is the tuple version of the classic default-list trap and many experienced developers miss it.

Q6. Is a tuple always hashable?

```
t = (1, [2, 3])  
hash(t)
```

Output

TypeError: unhashable type: 'list'

Now compare:

```
t = (1, (2, 3))  
hash(t)
```

Output

<some integer>

Why?

- A tuple is hashable only if all its elements are hashable.
- Hashability is recursive, not automatic.
- Correct interview answer:
 - “A tuple is conditionally hashable.”

Q7. Why does this unpacking permanently consume data?

```
g = (i for i in range(3))  
a, b, c = g  
print(list(g))
```

STAY TUNED...!

Output
[]

GENERATORS?

Why?

- Tuple unpacking pulls values eagerly.

Generators:

- Are consumed once
- Cannot be rewound
- Unpacking exhausts the **generator** entirely.

Q8. Why does this comparison feel unintuitive?

```
print((1, 2, 3) > (1, 2))  
print((1, 2) > (1, 2, 0))
```

Output: True False

Q9. Why does this closure behave unexpectedly?

```
t = (lambda: x for x in range(3))  
print([f() for f in t])
```

Output: [2, 2, 2]

Q10. Why does this identity check fail... but sometimes doesn't?

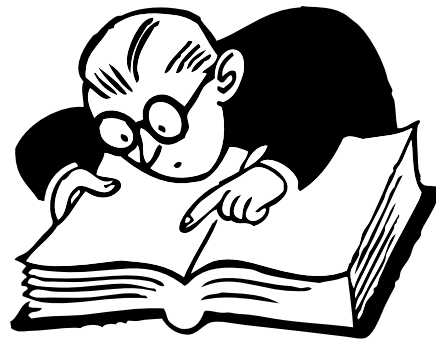
```
a = (1, 2)  
b = (1, 2)  
print(a is b)  
Output: False
```

Now:

```
a = ()  
b = ()  
print(a is b)  
Output: True
```

WHY???
**CHECK THE FIRST
3 COMMENTS**





Comment Questions



*Thank
You*